EMBEDDED LINUX SYSTEMS

ASSIGNMENT 02

Sensor Monitoring System (SMS)

Technical Report — Draft for Review

Student name	Le Tan Minh
Student ID	SE200684
Course	Embedded Linux Systems
Target platform	ARMv7 / QEMU VExpress-A9; host-side test utilities
Report status	Draft — verify marked gaps before final submission

This draft is based on the assignment brief and the test summaries supplied by the student. Results are distinguished from items that still require measurement or confirmation.

Abstract

This project implements a Sensor Monitoring System (SMS) for an embedded Linux environment. The intended system combines a kernel character-device driver with a multithreaded userspace application. The application reads simulated temperature and humidity samples, processes samples, sends alerts through POSIX message queues, and writes diagnostic messages to a log. The submission evidence supplied so far demonstrates host-side multithreaded tests, successful normal shutdown, log rotation, debugging of the worker threads, syscall tracing, performance-counter collection, and a 1,000-sample scheduling-jitter comparison.

The strongest quantitative real-time result in the supplied evidence is the host-side jitter comparison: SCHED_FIFO average jitter was 16,222.84 ns, compared with 70,758.65 ns under SCHED_OTHER. This corresponds to an average-jitter reduction of approximately 77.1% in that test. These measurements were explicitly taken on the Ubuntu host and must not be presented as QEMU-target measurements.

Important limitations remain. The Valgrind summary says the ARM-device path was unavailable during one run; the full output shows a separate host test executable and zero bytes in use at exit. The supplied perf artifact is a perf-stat summary, not a sampled `perf report` hotspot analysis. The strace artifact is a syscall trace and explanatory summary, not a `strace -c` top-five table. Memory-footprint measurements at T=5 s, T=30 s, and T=55 s have not been supplied.

Scope and evidence policy

This report does not invent measurements. Where the current evidence does not support a conclusion, the relevant section is marked as requiring a follow-up test. Replace the marked gaps with measured outputs before treating this document as a final submission.

Keywords

Embedded Linux; character device; POSIX threads; message queue; synchronization; Valgrind; GDB; strace; perf; real-time scheduling; jitter.

1. Requirements and Implementation Overview

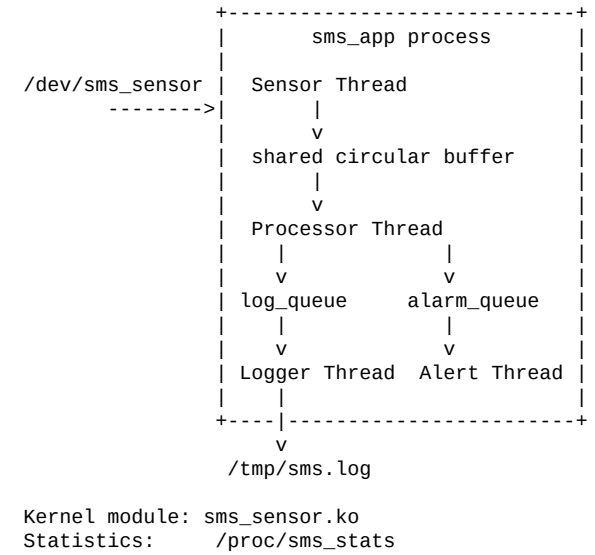
The assignment describes a system with a kernel sensor driver and four cooperating userspace threads: Sensor, Processor, Alert, and Logger. The driver is expected to expose `/dev/sms_sensor`, publish driver statistics through `/proc/sms_stats`, and support an interval-setting ioctl. The userspace program is expected to use a shared circular buffer, POSIX message queues, alert thresholds, log rotation, clean shutdown, and configurable command-line arguments.

Requirement area	Expected behavior	Evidence currently supplied
Kernel interface	Character device, proc statistics, interval ioctl	Driver source/module listed; QEMU module and device operation described in
Concurrency	Four threads, shared buffer and queues	GDB summary reaches all four thread functions; strace summary shows futex a
Logging	Timestamped log, flush, rotate above 1 MB	Log rotation test reports PASS and `sms.log.1` around 1 MB
Memory	No definite leaks after graceful shutdown	Host test Valgrind output: 11 allocations, 11 frees, 0 bytes in use
Debug/profiling	GDB session, strace analysis, perf hotspot	GDB summary, syscall trace, perf-stat counters supplied; some required analys
Real-time	SCHED_FIFO and jitter comparison	1,000 samples per policy recorded on Ubuntu host

Implementation file inventory supplied by the student includes `app/sms_app.c`, `app/sms_app.h`, `app/sms_threads.c`, `driver/sms_sensor.c`, `driver/sms_sensor.h`, and `driver/Makefile`. Several backup files and generated build artifacts are also present. The final archive should be reviewed for unnecessary backups and generated intermediate files.

2. System Architecture

The architecture separates device access, processing, alert delivery, and disk logging. The Sensor Thread is the device-facing producer. The Processor Thread consumes samples and evaluates rolling statistics and thresholds. Alerts are sent to an alarm message queue for the Alert Thread. Ordinary sample and status messages are sent to a log queue for the Logger Thread.



Data flow

• The driver generates simulated sensor readings and returns a CSV record to userspace.

• The Sensor Thread parses a record and attempts to publish it to the circular buffer.

• The Processor Thread consumes available samples, updates statistics, and evaluates alert conditions.

• The Alert Thread receives alert messages from the alarm queue and reports alerts.

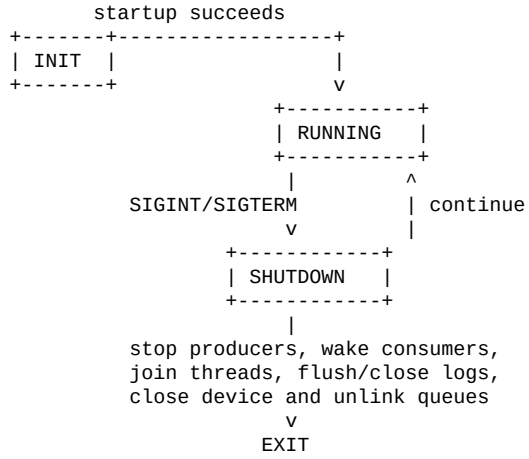
• The Logger Thread receives log messages and writes them to `/tmp/sms.log`; the log-rotation test checks the behavior when the file exceeds 1 MB.

Design rationale

A fixed-size circular buffer has bounded storage and avoids repeated allocation during steady-state sampling. A POSIX message queue provides message boundaries and blocking receive behavior for alerts. A separate Logger Thread prevents ordinary file I/O from being performed in the Sensor Thread, reducing the chance that disk activity delays sampling.

These are design rationales. Final submission should verify that the implementation’s actual queue names, buffer size, lock usage, and error paths match this description.

3. Application State Machine and Thread Responsibilities



Thread responsibilities

Thread	Primary work	Blocking / synchronization
Sensor	Read device periodically; publish sample	Periodic sleep; circular-buffer mutex/condition
Processor	Consume sample; calculate averages; detect alerts	Condition variable when buffer is empty; queue send
Alert	Receive and display alert messages	Blocking or timed POSIX message-queue receive
Logger	Receive log messages and write/rotate log	Message-queue receive and file I/O

The supplied GDB summary states that breakpoints were reached in `main`, `sensor_thread`, `processor_thread`, `alert_thread`, and `logger_thread`; `info threads` and `bt` were used; and the completed run exited normally. This supports that the host test build entered the main and worker-thread code paths.

Before final submission, attach the actual GDB transcript or a more complete session log. The summary alone does not show the values of the circular-buffer indices or the alert queue at a breakpoint.

4. Thread Synchronization and Deadlock Analysis

Shared data and expected protection

Shared data	Protection to document / verify
Circular-buffer storage, head, tail, size	One mutex guarding every read/write; condition variable used with the same mutex
Produced/consumed/overflow counters	Updated while holding the buffer mutex or with appropriate atomic operations
Shutdown flag	Signal-safe flag type such as `volatile sig_atomic_t` for signal notification; thread-visible state must follow the actual imp
Alert counters	Mutex or atomic updates if multiple threads can modify/read them
Logger file and rotation state	Owned by Logger Thread, or protected if accessed by other threads
POSIX message queues	Kernel queue synchronization; application still handles send/receive errors and shutdown messages

Lock-order analysis

A deadlock analysis should identify each mutex and the order in which locks may be acquired. A simple safe policy is to avoid acquiring a second application mutex while holding the circular-buffer mutex. The producer should hold the buffer mutex only while checking capacity, writing a sample, updating indices/counters, and signaling the consumer; it should release the mutex before logging or sending a message to another queue.

The evidence supplied does not include a complete lock-order map from source inspection, so this report cannot claim a formal proof that all lock cycles are impossible. Complete the table below from the actual code.

Operation	Locks acquired, in order	Released before blocking I/O?
Sensor publishes sample	[VERIFY FROM SOURCE]	[VERIFY]
Processor consumes sample	[VERIFY FROM SOURCE]	[VERIFY]
Processor sends alert/log	[VERIFY FROM SOURCE]	[VERIFY]
Shutdown and thread join	[VERIFY FROM SOURCE]	[VERIFY]

Potential race condition

A common race risk is reading the buffer's `size`, `head`, or `tail` outside the mutex while another thread updates them. Another risk is accessing shared alert counters concurrently. The final report should describe a real issue encountered during development and point to the specific protection or code change that fixed it; do not claim a bug was found and fixed unless this happened.

5. Kernel Driver and Userspace Integration

The project contains `driver/sms_sensor.c`, `driver/sms_sensor.h`, a Makefile, and a built `sms_sensor.ko`. Earlier QEMU test notes record successful module loading, creation of `/dev/sms_sensor`, and readable `/proc/sms_stats` output. The module emitted the standard warning that loading an out-of-tree module taints the kernel; that warning by itself is not a load failure.

Expected driver contract

Interface	Purpose	Verification evidence needed
/dev/sms_sensor	Read CSV timestamp, temperature, humidity	Capture one or more actual read records from QEMU
/proc/sms_stats	Expose read count, latest readings and uptime	Capture before/after output in logs
SMS_SET_INTERVAL ioctl	Set minimum read interval in milliseconds	Show ioctl success against the actual device
Module lifecycle	Load and unload without oops	Capture `insmod`, `lsmod`, `dmesg`, and clean `rmmod`

Integration limitation

The supplied host-side tests use a regular CSV file instead of the actual sensor device. Both the Valgrind summary and strace/perf summaries note that the ioctl error is expected for this host simulator. Therefore those runs validate userspace behavior against test data but do not prove the real device ioctl path.

For final evidence, record `/proc/sms_stats` before and after a 60-second QEMU run at the requested 500 ms interval. The assignment expects approximately 120 reads, with tolerance of ±10. Do not report this target as achieved until the actual before/after outputs are captured.

6. Memory Management and Valgrind

Observed Valgrind result

Metric	Observed host-test result
Executable	`test_host/sms_app_test`
Total heap allocations	11
Total heap frees	11
Bytes in use at exit	0 bytes in 0 blocks
Error summary	0 errors from 0 contexts

The full Valgrind output supplied reports that all heap blocks were freed and no leaks were possible for that completed host-test run. The separate summary file also records a run in which `/dev/sms_sensor` was unavailable and the application exited early. These are different test conditions and should not be conflated.

The result supports the narrow conclusion that the recorded host test completed with zero heap bytes in use at exit. It does not by itself prove the full ARM/QEMU runtime has no leaks, nor does it establish that the program ran under Valgrind for the required 30 seconds with SIGINT.

Required follow-up

• Run the x86_64 test build for at least 30 seconds and terminate it with SIGINT.

• Save the complete output to `logs/valgrind_sms.txt` or the exact filename required by the assignment.

• Verify `definitely lost: 0 bytes` and explain any `possibly lost` or `still reachable` entries.

• Record the exact command, executable, duration, exit code, and whether the actual sensor interface or a CSV simulator was used.

Memory footprint

The required VmRSS and VmSize samples at T=5 s, T=30 s, and T=55 s were not included in the supplied output. Insert measured values here after collecting `/proc/PID/status` for the running QEMU-target process or the assignment-approved process environment. Do not substitute Valgrind heap totals for VmRSS/VmSize; they measure different things.

Time	VmRSS	VmSize	Interpretation
T=5 s	[MEASURE]	[MEASURE]	[ADD ANALYSIS]
T=30 s	[MEASURE]	[MEASURE]	[ADD ANALYSIS]
T=55 s	[MEASURE]	[MEASURE]	[ADD ANALYSIS]

7. Debugging with GDB

The supplied GDB summary records breakpoints in `main`, `sensor_thread`, `processor_thread`, `alert_thread`, and `logger_thread`. It also states that `info threads` was used to inspect active threads, `bt` was used to inspect stack traces, and the application exited normally in the completed run.

What this demonstrates

• The debugger could load the host debug executable and stop in the main function.

• The test execution reached all four worker-thread functions.

• Thread inspection and stack-trace commands were used during the session.

• The CSV test input caused an ioctl warning because a regular file does not implement the sensor device ioctl; the summary identifies this as expected in the host test.

What is not yet demonstrated by the summary

The assignment asks for a complete session showing either a race-condition investigation with buffer indices and all-thread backtraces, or an alert-logic investigation that modifies a temperature value and observes the alarm path. The supplied summary does not contain the full command transcript, variable values, or the observed queue message. Attach the original GDB transcript and explain one specific observation.

Recommended transcript excerpt

```
(gdb) break main
(gdb) break sensor_thread
(gdb) break processor_thread
(gdb) break alert_thread
(gdb) break logger_thread
(gdb) run
(gdb) info threads
(gdb) thread apply all bt
```

The commands above are a recommended format for documenting a real session, not a claim that every line was present in the supplied transcript. Include only commands actually run and their actual output in the final evidence.

8. System-Call Analysis with strace

The supplied strace summary says the test was run with `strace -f -tt -o logs/strace_sms_app.log ./test_host/sms_app_test -i 100 -t 3 -v`. The trace shows the application creating and using POSIX message queues, synchronizing threads, reading the CSV input, writing output/log data, and cleaning up at shutdown.

System call / API family	Evidence / role in this application
openat	Opens the sensor CSV and log file in the host test
read	Reads sensor records from the CSV input
write	Writes sample/status output and log data
futex	Underlying synchronization used by pthread mutexes/condition variables on Linux
clock_nanosleep	Implements periodic waiting
mq_open	Creates or opens POSIX message queues
mq_timedsend	Sends sample, log, or shutdown messages
mq_timedreceive	Receives messages and shutdown markers
mq_unlink / close	Cleans up queue names and file descriptors

Interpretation

The trace confirms that both synchronization and message passing are active in the host test. Periodic waiting appears through `clock_nanosleep`; queue sends and receives support the producer/consumer design. File reads are attributable to the CSV simulator, not the kernel sensor device.

The required `strace -c` top-five syscall counts and percentage-of-time summary have not been supplied. A timestamped full trace cannot substitute for a syscall-count table. Likewise, user-mode versus kernel-mode time should not be inferred from this trace alone.

Follow-up

```
strace -f -c -o logs/strace_sms.txt ./test_host/sms_app_test -i 100 -t 3 -v
```

Run the command in the host environment where the test executable works, then use the resulting count and time columns to list the top five calls and explain each one.

9. Performance Profiling with perf

Available perf-stat measurements

Counter	Recorded value
Elapsed time	3.011128826 seconds
Task clock	8.37 msec
Context switches	105
CPU migrations	13
Page faults	81
User time	0.004654 seconds
System time	0.006982 seconds

These values come from the supplied host-side CSV simulator run. The recorded note explicitly says they are not QEMU measurements. The program ran for about three seconds and exited normally.

Interpretation and limitation

The task-clock total is much smaller than elapsed wall time because the application spends much of its run waiting for periodic events or synchronization rather than continuously consuming a CPU. The observed context switches are consistent with a multithreaded application that waits and wakes. However, these counters do not identify the function with the largest CPU cost.

The assignment specifically asks for `perf record -g` followed by `perf report`, a hotspot identification, and a concrete optimization suggestion. A `perf stat` summary is useful supplementary evidence but does not satisfy the sampled call-graph requirement.

Required follow-up

```
sudo perf record -g -- ./test_host/sms_app_test -i 100 -t 30 -v
sudo perf report
```

Save the actual report output as `logs/perf_sms.txt`. Identify the hottest function using the recorded percentages, then base the optimization proposal on that output rather than guessing.

10. Real-Time Scheduling and Jitter

The supplied comparison reports 1,000 jitter samples per policy, a 1 ms period, `CLOCK_MONOTONIC`, absolute `clock_nanosleep` timing, and `mlockall(MCL_CURRENT | MCL_FUTURE)`. The measurement was performed on the Ubuntu host, not in QEMU.

Metric	SCHED_OTHER	SCHED_FIFO (priority 50)
Samples	1,000	1,000
Minimum jitter	52,906 ns	2,565 ns
Average jitter	70,758.65 ns	16,222.84 ns
Maximum jitter	990,504 ns	99,551 ns
P99 jitter	[NOT PROVIDED]	[NOT PROVIDED]

Average-jitter reduction is calculated as (70,758.65 − 16,222.84) / 70,758.65 × 100 ≈ 77.1%. The recorded maximum jitter also decreased from 990,504 ns to 99,551 ns in this test. These results suggest that SCHED_FIFO improved timing consistency under the measured host conditions.

P99 is not present in the supplied summary and must be computed from the CSV values. The assignment also asks for a text histogram using four bins: 0–100 µs, 100–500 µs, 500–1,000 µs, and greater than 1 ms. Those bin counts were not included in the supplied comparison.

Interpretation

The measurement period was 1 ms for the jitter test; this is distinct from the application’s default 500 ms sensor sampling period. The report must keep those two periods separate. Because these are host measurements, they cannot be represented as real-time guarantees for the QEMU target or production hardware.

11. Real-Time Analysis and Improvement Plan

Does SCHED_FIFO improve jitter?

In the supplied host test, yes: average jitter decreased by approximately 77.1%, and maximum jitter decreased from 990,504 ns to 99,551 ns. P99 comparison cannot yet be answered because P99 values are missing. The result depends on system load, privileges, kernel configuration, CPU scheduling, and the measurement method.

Acceptable jitter

The assignment asks the student to state an acceptable jitter threshold for a 500 ms industrial sensor period and assess whether the observed result meets it. The threshold must be justified by the application's requirements; the assignment does not provide one universal threshold. The current measurements use a 1 ms test period on the host, so they should not be directly used to certify a 500 ms target application.

Potential improvements

• Use a PREEMPT_RT-enabled kernel if the deployment requires stronger scheduling-latency guarantees.

• Consider CPU affinity or CPU isolation for the timing-sensitive sensor thread, after measuring the impact on the target platform.

• Keep blocking file I/O and expensive processing out of the sensor thread.

• Preallocate buffers and avoid dynamic allocation in the periodic path.

• Record wake-up latency on the actual target and under representative system load.

Missing real-time evidence

Add the P99 value for both policies, a histogram count for each required bin, and the command/output showing whether `sched_setscheduler()` succeeded. Also state whether the test process had the required privileges for SCHED_FIFO.

12. Logging, Shutdown, and Operational Tests

Log rotation

The supplied log-rotation summary describes a QEMU ARMv7 / Linux 5.15.0 test. A 1,025 KiB file was created at `/tmp/sms.log`; after running the application, the old log was renamed to `/tmp/sms.log.1` at approximately 1.0 MiB and a new `/tmp/sms.log` of approximately 2.3 KiB existed. The summary marks this test PASS.

Graceful shutdown

The project notes show a successful SIGINT test after the driver was loaded: the application printed `SMS application stopped.` and returned exit code 0. The host strace trace also shows queue cleanup (`mq_unlink`), file closure (`close`), and normal process termination (`exit_group(0)`).

Operational checklist

Test	Current status based on supplied evidence
Host test exits normally	PASS in GDB/strace/perf summaries
SIGINT graceful stop	PASS in reported test after driver load
Log rotation above 1 MB	PASS according to evidence/log_rotation_test.txt
Driver read-count before/after 60 seconds	Not supplied in the output shown
Run app for 60 seconds with all four threads	Not proven by the excerpts shown
Final demo video	No file listed under demo/

The demo folder was empty when checked. A screen recording of 3–5 minutes is still required by the assignment. The demo should show QEMU boot, driver/device/proc entries, verbose samples, an alert, log output, and Ctrl+C shutdown.

13. Reflection

The project combines several topics that are easy to test individually but harder to integrate reliably: kernel/userspace interfaces, periodic sampling, synchronization, message queues, logging, and shutdown. A useful lesson from the current evidence is that a successful host simulation is not identical to a successful target-device test. For example, the CSV simulator supports testing the application's threads and queues, but it cannot validate a device-specific ioctl.

If redesigning the system, I would make the sensor interface configurable, keep the periodic sensor path minimal, and make the host simulator and real driver implement the same small interface contract. I would also standardize the evidence collection scripts so every test saves its exact command, output, environment, duration, and exit status.

The skills most likely to transfer to embedded Linux work are reading driver interfaces, using synchronization primitives correctly, investigating threads with GDB, measuring system calls and performance, and validating real-time behavior instead of assuming it. A future improvement could publish selected readings and alerts through MQTT to a dashboard, while keeping local logging available when the network is down.

Conclusion

The supplied evidence demonstrates meaningful progress: the driver and application artifacts exist; the host test reaches all four thread functions; the recorded Valgrind host test ends with zero bytes in use and zero errors; the trace shows message-queue and synchronization activity; log rotation passed in QEMU; and host-side SCHED_FIFO jitter was lower than SCHED_OTHER in the recorded test.

The project is not yet fully evidenced against every rubric item. Before final submission, add the missing memory-footprint table, `strace -c` top-five analysis, sampled `perf report`, complete GDB transcript, jitter P99 and histogram, actual `/proc/sms_stats` before/after run, and the 3–5 minute demo video. Keep the distinction between host-side and QEMU results explicit.

References

1. ASSIGNMENT02_EmbeddedLinux.docx, Embedded Linux Systems – Capstone Assignment, Assignment 02, pages 1–15.

2. Linux man-pages project: pthreads, POSIX message queues, `clock_nanosleep(2)`, `sched_setscheduler(2)`, `mlockall(2)`, `strace(1)`, `perf(1)`, and Valgrind documentation. Add the exact versions and access dates used during the final tests.

END OF DRAFT — verify all bracketed gaps and attach real evidence before submission.